

Bài thực hành số 3

XÂY DỰNG ỨNG DỤNG VỚI FREERTOS VÀ TOUCHGFX

1. Mục tiêu

- Lập trình với hệ điều hành FreeRTOS.
- Lập trình xây dựng ứng dụng với framework TouchGFX.
- Xây dựng ứng dụng đa chức năng:
 - + Ghép nối ngoại vi với STM32F4.
 - + Giao diện đồ họa, thực thi đa nhiệm với FreeRTOS và TouchGFX.

2. Chuẩn bị

- Tài liệu thực hành, Kit STM32F429I-DISC1, mini-USB cable.
- Các phần mềm: STM32CubeIDE 1.17.0, TouchGFX 4.24.1 hoặc 4.25.0, Hercules.

3. Nội dung

3.0. Làm quen với FreeRTOS

- Mở STM32CubeIDE, tạo project mới.
- Cấu hình các thành phần ngoại vi.
 - + Cấu hình PG13 và PG14 là Output.
 - + Cấu hình RCC và System clock để CPU chạy ở tần số 90 MHz.
 - + Cấu hình USART1 ở chế độ Asynchronous 115200 bps.
- Cấu hình FreeRTOS trong Middleware.
 - + Chọn Interface là CMSIS V2.
 - + Thêm task myTask02.
- Sinh code và lập trình các hàm của 2 task:
 - + defaultTask
 - + myTask02



```

295 /* USER CODE END Header_StartDefaultTask */
296 void StartDefaultTask(void *argument)
297 {
298     /* USER CODE BEGIN 5 */
299     /* Infinite loop */
300     for(;;)
301     {
302         osDelay(500);
303         HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
304     }
305     /* USER CODE END 5 */
306 }

```

```

314 /* USER CODE END Header_StartTask02 */
315 void StartTask02(void *argument)
316 {
317     /* USER CODE BEGIN StartTask02 */
318     /* Infinite loop */
319     for(;;)
320     {
321         HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
322         HAL_UART_Transmit(&uart1, "Hello from user task\n", 21, 10);
323
324         osDelay(500);
325     }
326     /* USER CODE END StartTask02 */
327 }

```

- Dịch, nạp code, chạy chương trình trên board STM32F429I.

- Quan sát kết quả chạy chương trình:

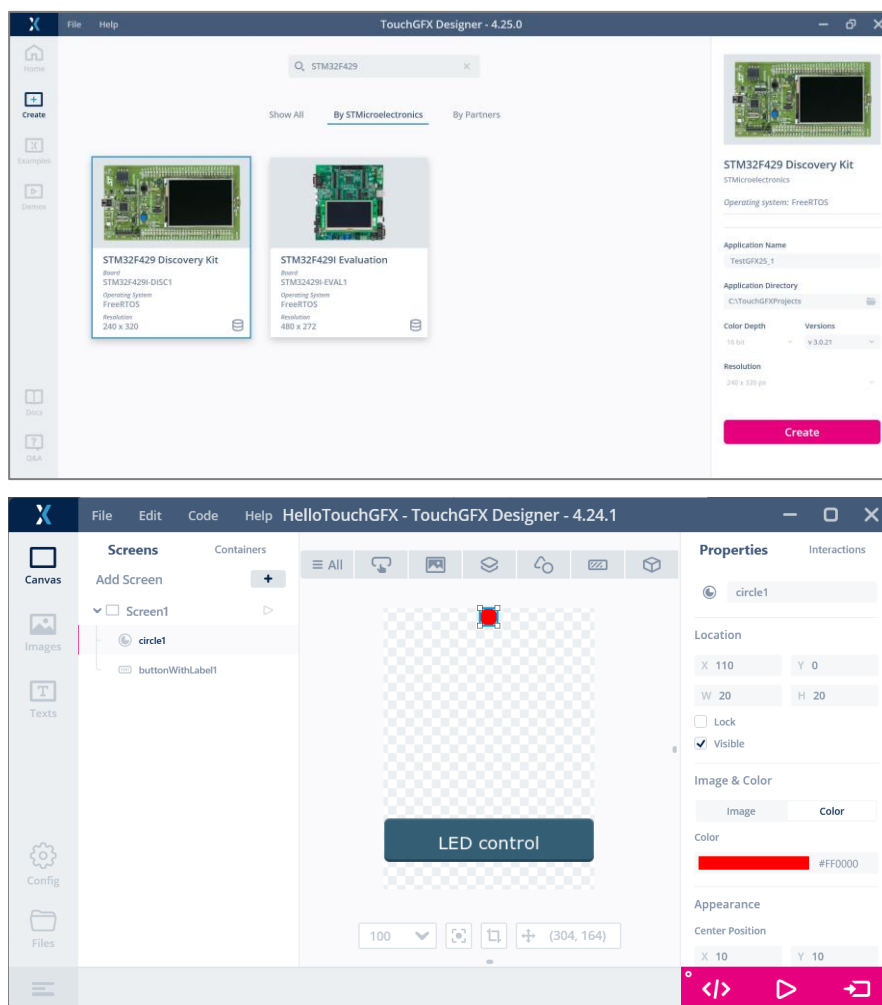
+ Đèn LED trên chân PG13 nhấp với tần số 1 Hz.

+ Dòng chữ “Hello from user task” được gửi về PC và hiển thị trên Hercules.

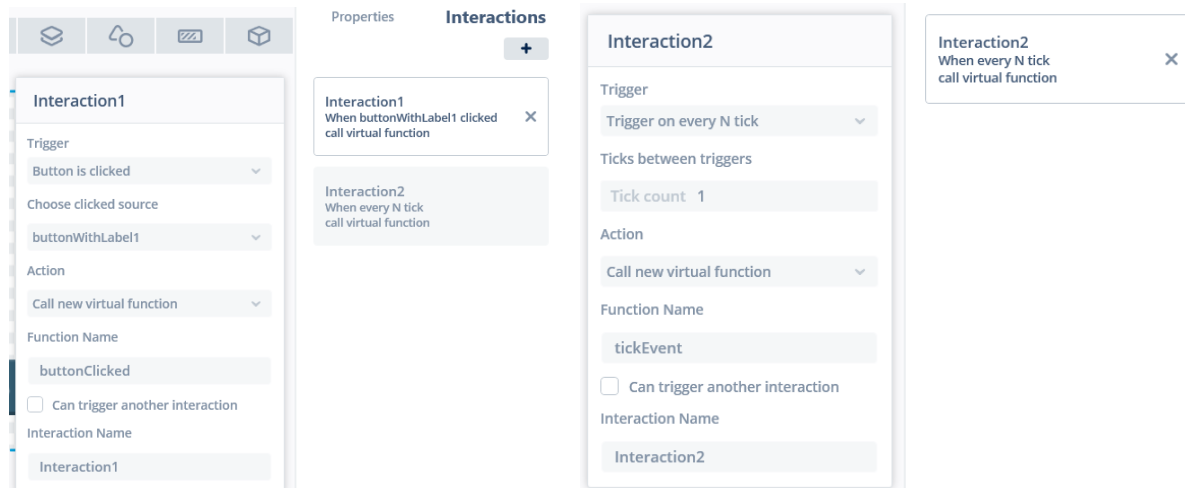
3.1 Làm quen với TouchGFX và FreeRTOS

3.1.1 Tạo project

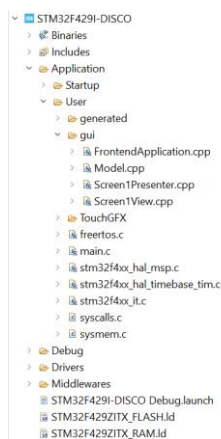
- Mở TouchGFX Designer, tạo project mới với board **STM32F429**. Đặt tên project là HelloTouchGFX. Chú ý chọn version 3.0.21.



- Trong phần Canvas để thiết kế giao diện, đặt 1 đối tượng Circle lên Screen1. Thiết lập các thông số màu sắc, kích thước cho circle1 như hình. Chú ý: set Radius = 10.
- Đặt đối tượng Button with label lên Screen1. Đặt text là LED control.
- Thêm + thiết lập hàm xử lý sự kiện **clicked** của button trong phần **Interaction**. Hàm buttonClicked() sẽ được gọi mỗi khi nút được bấm.



- Thêm + thiết lập hàm xử lý sự kiện **thời gian** của Screen1. Hàm tickEvent() sẽ được gọi ứng với mỗi sự kiện tick cập nhật thời gian của Screen1.
- Bấm nút Generate code (F4) màu đỏ góc dưới phải.
- Double click file **.project** trong thư mục **HelloTouchGFX\STM32CubeIDE** để import project vào STM32CubeIDE.



- Dịch và nạp chương trình lên kit STM32F429. Kết quả thu được là hình tròn đỏ và nút bấm hiện trên màn hình đúng vị trí thiết kế trên TouchGFX.
- Tìm hiểu cấu trúc project.

3.1.2.Lập trình ghép nối và xử lý sự kiện

- Vào file main.c, thêm code khởi tạo cấu hình các chân PG13, PG14, và PA0 (đèn LED và nút bấm).
- Khai báo thêm thành viên cho Screen1View.
 - + Override các hàm buttonClicked() và tickEvent() đã khai báo ở lớp ViewBase.
 - + Thêm biến tickCount.

```

Screen1View.cpp  Screen1View.hpp  Screen1ViewBase.hpp
1  #ifndef SCREEN1VIEW_HPP
2  #define SCREEN1VIEW_HPP
3
4  #include <gui_generated/screen1_screen/Screen1ViewBase.hpp>
5  #include <gui/screen1_screen/Screen1Presenter.hpp>
6
7  class Screen1View : public Screen1ViewBase
8  {
9  public:
10     Screen1View();
11     virtual ~Screen1View() {}
12     virtual void setupScreen();
13     virtual void tearDownScreen();
14
15     void buttonClicked();
16     void tickEvent();
17 protected:
18     uint32_t tickCount = 0;
19 };
20
21 #endif // SCREEN1VIEW_HPP
22
23 /* USER CODE BEGIN MX_GPIO_Init_2 */
24 /*Configure LED pins : PG13 PG14 */
25 GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_14;
26 GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
27 GPIO_InitStruct.Pull = GPIO_NOPULL;
28 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
29 HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);
30
31 /*Configure GPIO pin : PA0 */
32 GPIO_InitStruct.Pin = GPIO_PIN_0;
33 GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
34 GPIO_InitStruct.Pull = GPIO_NOPULL;
35 HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
36 /* USER CODE END MX_GPIO_Init_2 */

```

- Xử lý sự kiện tick() để cập nhật vị trí hình tròn.
- Xử lý sự kiện bấm nút để bật/tắt đèn LED PG13.

Chú ý: #include “touchgfx/widgets/canvas/Circle.hpp”

```

Screen1View.cpp  Screen1View.hpp  Screen1ViewBase.hpp
26 void Screen1View::buttonClicked()
27 {
28     HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
29 }
30
31 void Screen1View::tickEvent()
32 {
33     tickCount += 2;
34     tickCount = tickCount % 240;
35
36     //draw circle at the new position
37     circle1.moveTo(tickCount, 50);
38     circle1.invalidate();
39 }

```

- Dịch và nạp chương trình lên kit, quan sát hoạt động.
 - + Hình tròn đỏ chạy lặp lại ngang màn hình.
 - + Khi bấm nút LED control thì đèn PG13 (LED xanh) bật/tắt.

3.1.3 Lập trình ghép nối nút bấm USER_BUTTON

Khai báo queue trong phần Private Variable file main.c (sau khai báo tạo thread ID đã có)

```
/* USER CODE BEGIN PV */
osMessageQueueId_t myQueue01Handle;
const osMessageQueueAttr_t myQueue01_attributes = {
    .name = "myQueue01"
};
/* USER CODE END PV */
```

Tạo queue trước khi tạo task trong file main.c

```
/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
myQueue01Handle = osMessageQueueNew (16, sizeof(uint16_t), &myQueue01_attributes);

/* USER CODE END RTOS_QUEUES */
```

Liên tục polling trạng thái nút bấm trong **default task**, gửi dữ liệu vào queue

```
void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
        {
            uint8_t data = 'X';
            osMessageQueuePut(myQueue01Handle, &data, 0, 10);
        }

        osDelay(1);
    }
    /* USER CODE END 5 */
}
```

Hiệu chỉnh hàm **tickEvent** của Screen1 View để nhận dữ liệu từ queue và cập nhật animation. Chú ý: khai báo extern myQueue01Handle.

```
void Screen1View::buttonClicked()
{
    HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
}

extern osMessageQueueId_t myQueue01Handle;
void Screen1View::tickEvent()
{
    tickCount += 2;
    tickCount = tickCount % 240;

    float x = tickCount / 55.0f;
    float y = sin(x)+sin(2*x)+sin(3*x)+1;

    uint8_t res;
    if (osMessageQueueGetCount(myQueue01Handle) > 0)
    {
        osMessageQueueGet(myQueue01Handle, &res, NULL, osWaitForever);
        if (res == 'X')
        {
            circle1.moveTo((int16_t)floor(x*55),200-(int16_t)floor(y*50));
            circle1.invalidate();
        }
    }
}
```

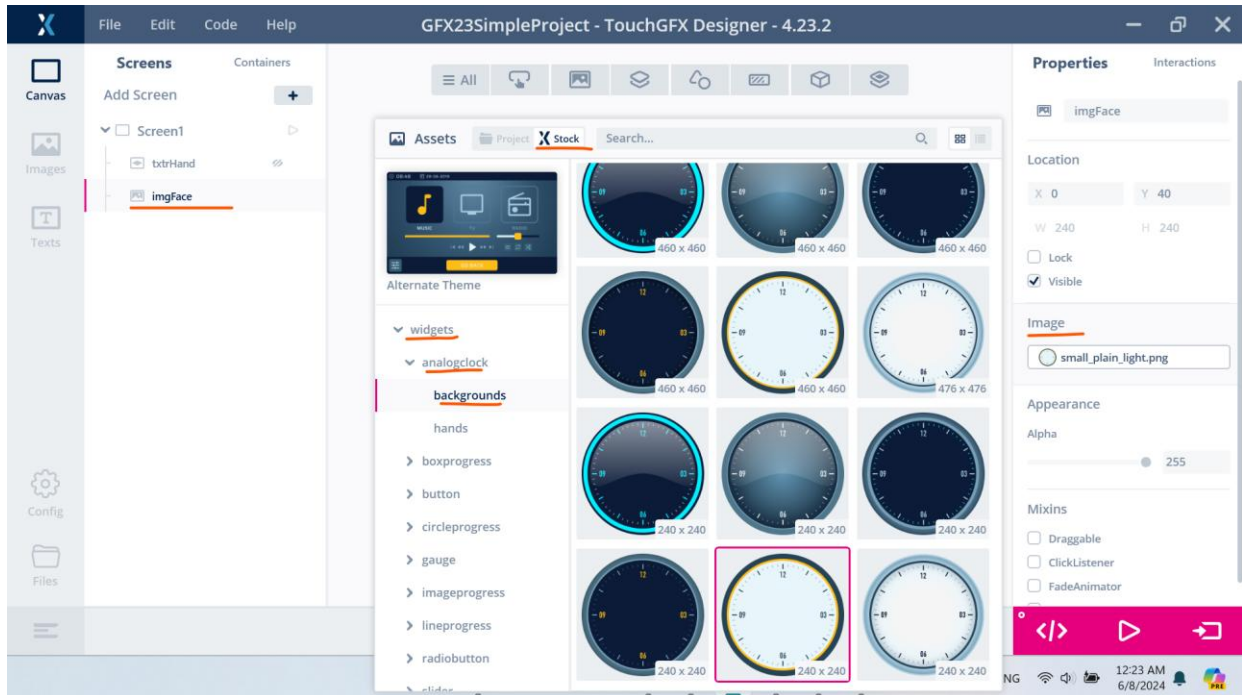
Dịch và chạy chương trình trên kit. Quan sát hoạt động của mạch:

- Khi bấm nút thì hình tròn di chuyển từ trái sang phải theo quỹ đạo dao động lên xuống.

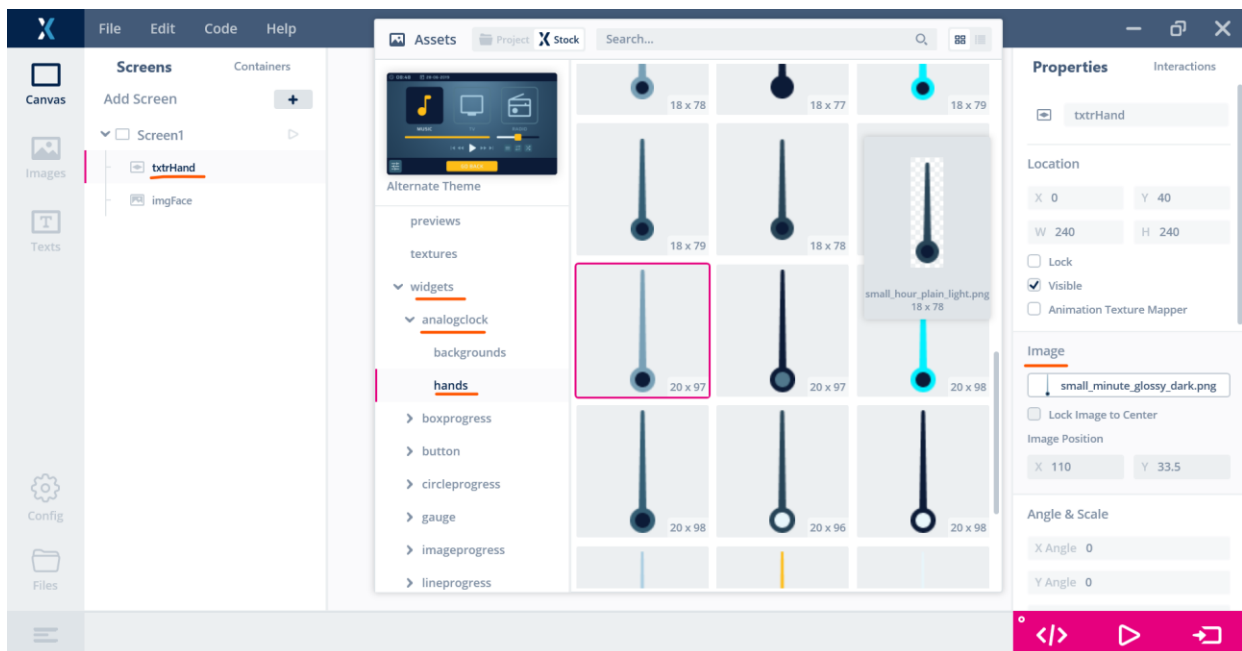
Giải thích hoạt động của chương trình.

3.2 Xây dựng ứng dụng đồng hồ bấm giây

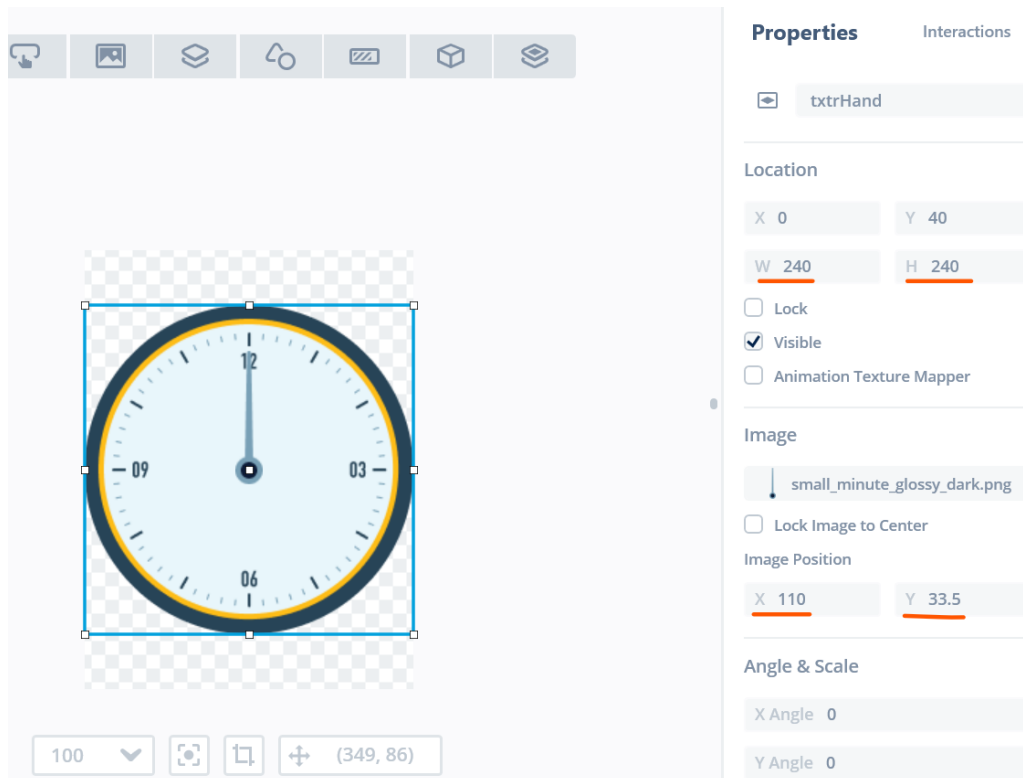
- Mở TouchGFX Designer, tạo project mới với board STM32F429. Đặt tên project tùy chọn.
- Trong phần Canvas để thiết kế giao diện, đặt 1 đối tượng Image (trong mục Images) lên Screen1 để hiển thị mặt đồng hồ. Chọn ảnh watch face cho đối tượng trong mục Image → Assets → Stock.



- Đặt thêm 1 đối tượng Texture Mapper (cũng trong mục Images) lên Screen1 để hiển thị kim đồng hồ. Chọn ảnh watch hand cho đối tượng trong mục Image.



- Thiết lập thông số cho kim đồng hồ txtrHand. Có thể thử thay đổi thông số Z Angle xem kim quay đúng hay chưa.



- Bấm nút Generate code (F4) màu đỏ góc dưới phải.
- Double click file **.project** trong thư mục **[Project folder]\STM32CubeIDE** để import project vào STM32CubeIDE. Chú ý: cần delete project cũ (nhưng không xóa thư mục trên đĩa).
- Dịch và nạp chương trình lên kit STM32F429. Kết quả thu được là hình mặt đồng hồ có một kim.
- Tìm hiểu cấu trúc project.
- Lập trình để kim chạy.

Khai báo thêm thành viên cho Screen1View

```

1 #ifndef SCREEN1VIEW_HPP
2 #define SCREEN1VIEW_HPP
3
4 #include <gui_generated/screen1_screen/Screen1ViewBase.hpp>
5 #include <gui/screen1_screen/Screen1Presenter.hpp>
6
7 class Screen1View : public Screen1ViewBase
8 {
9 public:
10     Screen1View();
11     virtual ~Screen1View() {}
12     virtual void setupScreen();
13     virtual void tearDownScreen();
14     void handleTickEvent();
15     uint32_t tickCount;
16 protected:
17 };
18
19 #endif // SCREEN1VIEW_HPP
20

```

Xử lý sự kiện tick() để cập nhật vị trí kim đồng hồ. Chú ý khởi tạo giá trị = 0 cho tickCount ở hàm tạo của Screen1View.

```
void Screen1View::handleTickEvent()
{
    Screen1ViewBase::handleTickEvent();
    tickCount++;
    float rad = (tickCount % 360) * 3.14f / 180;

    //update and invalidate the clock hand
    txtrHand.updateZAngle(rad);
}
```

Dịch và nạp chương trình lên kit, quan sát chuyển động của kim đồng hồ.

Tự làm: sử dụng hàm

uint32_t osKernelGetTickCount (void)

Ref: [Kernel Information and Control \(arm-software.github.io\)](https://arm-software.github.io)

để lấy thời gian hệ thống, sau đó tính và cập nhật góc dịch chuyển của kim theo tham số giây.

Chú ý: #include <cmsis_os.h>

- Lập trình ghép nối nút bấm USER_BUTTON để bật/tắt bấm giờ.

Cấu hình chân vào ra trong hàm MX_GPIO_Init() ở main.c

```
/*Configure GPIO pin : USER_BUTTON is A0 pin */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

Khai báo queue trong phần Private Variable file main.c (sau khai báo tạo thread ID đã có)

```
/* USER CODE BEGIN PV */
osMessageQueueId_t Queue1Handle;
const osMessageQueueAttr_t Queue1_Attributes = {
    .name = "Queue1"
};
osStatus_t r_state;
/* USER CODE END PV */
```

Tạo queue trước khi tạo task trong file main.c

```
205 /* USER CODE BEGIN RTOS_QUEUES */
206 /* creation of Queue1 */
207 Queue1Handle = osMessageQueueNew (8, sizeof(uint8_t), &Queue1_attributes);
208 /* USER CODE END RTOS_QUEUES */
~~~
```

Liên tục polling trạng thái nút bấm trong default task, gửi dữ liệu vào queue

```
void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
        {
            uint32_t count = osMessageQueueGetCount(Queue1Handle);
            if (count < 2)
            {
                uint8_t data = 'P';
                osMessageQueuePut(Queue1Handle, &data, 0, 10);
            }
        }
        osDelay(1);
    }
}
```


Hiệu chỉnh hàm `handleTickEvent` của `Screen1View` để nhận dữ liệu từ queue và cập nhật hoạt động của kim đồng hồ.

```
void Screen1View::handleTickEvent()
{
    static uint32_t prevTick = 0;
    Screen1ViewBase::handleTickEvent();
    tickCount++;
    float rad = 0;

    uint8_t res = 0;
    uint32_t count = osMessageQueueGetCount(Queue1Handle);
    if (count > 0)
    {
        osMessageQueueGet(Queue1Handle, &res, NULL, osWaitForever);
        if (res == 'P')
        {
            //update and invalidate the clock hand
            rad = ((tickCount - prevTick) % 360) * 3.14f / 180;
            txtrHand.updateZAngle(rad);
        }
    }
    else
    {
        prevTick = tickCount;
    }
}
```

Dịch và chạy chương trình trên kit. Quan sát hoạt động của mạch:

- Khi không bấm nút thì kim đồng hồ đứng yên.
- Khi bấm nút thì kim đồng hồ chạy theo chiều kim đồng hồ, cho tới khi nhả nút.

Giải thích hoạt động của chương trình.

3.4 Bài tập tự làm

Sử dụng mã nguồn cho trong project “SimpleRacing”, xây dựng giao diện và phần hoạt hình cơ bản cho 1 game đua xe.

- Đường đua di chuyển theo chiều từ trên xuống.
- Xe chỉ có thể di chuyển theo chiều ngang.
- Ghép nối phần cứng để điều khiển xe. Có thể tùy chọn 1 trong 2 phương án:
 - o Tự làm mạch có 2 nút bấm nối với chân PG2 và PG3 để điều khiển xe dịch chuyển sang trái/phải. Chú ý: cần cấu hình PG2 và PG3 ở chế độ input có pull-up resister.
 - o Dùng joystick để điều khiển xe dịch chuyển trái phải. Chỉ cần ghép nối 1 kênh ADC vì chỉ cần điều khiển xe theo 1 phương ngang.
- Thêm các tính năng tăng tốc, sinh chương ngại vật ngẫu nhiên, tính điểm, lưu và hiển thị High score.

Viết báo cáo mô tả chi tiết hoạt động của phần cứng + phần mềm trong ứng dụng game.

Chú ý: project mẫu cần được để trong thư mục C:\TouchGFXProjects.

- Hết -